

Code Explanation

Delta Live Tables Implementation

This code is used to create a Delta Live Table (DLT) pipeline that ingests customer and order data from CSV files, applies transformations, and creates two tables: `ordersummary` and `customeraggregatespend`. The `ordersummary` table is a Slowly Changing Dimension (SCD) Type 2 table that tracks changes to customer data over time.

Overview

The code defines a DLT pipeline that reads customer and order data from CSV files, applies data quality checks, and creates two tables. The `ordersummary` table is an SCD Type 2 table that captures changes to customer data, while the `customeraggregatespend` table aggregates customer spending by name and date.

Step 1: Define the Catalog and Schema Names

This step defines the catalog and schema names used in the DLT pipeline.

```
CATALOG = "gen_ai_poc_databrickscoe"
SCHEMA = "sdlc_wizard"
```

Step 2: Define Schemas for Source Data

This step defines the schemas for the customer and order data.

```
customer_schema = StructType([
    StructField("CustId", StringType(), True),
    StructField("Name", StringType(), True),
    StructField("EmailId", StringType(), True),
    StructField("Region", StringType(), True)
])

order_schema = StructType([
    StructField("OrderId", StringType(), True),
    StructField("ItemName", StringType(), True),
    StructField("PricePerUnit", DoubleType(), True),
    StructField("Qty", IntegerType(), True),
    StructField("Date", DateType(), True),
    StructField("CustId", StringType(), True)
])
```

Step 3: Create Source Tables

This step creates two source tables, `customer` and `order`, by reading CSV files and applying data quality checks.

```
@dlt.table(
    name="customer",
```

```

        comment="Raw customer data loaded from CSV"
    )
def customer():
    return (
        spark.read.schema(customer_schema)
        .csv("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/customer
data")
        .filter(col("CustId").isNotNull() &
                col("Name").isNotNull() &
                col("EmailId").isNotNull() &
                col("Region").isNotNull())
        .dropDuplicates()
    )
@dlt.table(
    name="order",
    comment="Raw order data loaded from CSV with TotalAmount calcula
ted"
)
def order():
    return (
        spark.read.schema(order_schema)
        .csv("/Volumes/gen_ai_poc_databrickscoe/sdlc_wizard/orderdat
a")
        .filter(col("OrderId").isNotNull() &
                col("ItemName").isNotNull() &
                col("PricePerUnit").isNotNull() &
                col("Qty").isNotNull() &
                col("Date").isNotNull() &
                col("CustId").isNotNull())
        .dropDuplicates()
        .withColumn("TotalAmount", col("PricePerUnit") * col("Qty"))
    )

```

Step 4: Implement SCD Type 2 for ordersummary Table

This step creates the `ordersummary` table, an SCD Type 2 table that captures changes to customer data.

```

@dlt.table(
    name="ordersummary",
    table_properties={
        "delta.autoOptimize.optimizeWrite": "true",
        "delta.autoOptimize.autoCompact": "true"
    },
    comment="SCD Type 2 table containing customer and order data",
    temporary=False,
    schema=StructType([
        StructField("CustId", StringType(), True),
        StructField("Name", StringType(), True),
        StructField("EmailId", StringType(), True),
        StructField("Region", StringType(), True),
        StructField("OrderId", StringType(), True),
        StructField("ItemName", StringType(), True),
        StructField("PricePerUnit", DoubleType(), True),
        StructField("Qty", IntegerType(), True),
    ])
)

```

```

        StructField("Date", DateType(), True),
        StructField("TotalAmount", DoubleType(), True),
        StructField("IsActive", BooleanType(), True),
        StructField("StartDate", TimestampType(), True),
        StructField("EndDate", TimestampType(), True)
    ])
)
def ordersummary():
    # Get the current and previous customer data
    current_customer = dlt.read("customer")

    # Get the order data
    orders = dlt.read("order")

    # Join current customer and order data
    current_joined = (
        current_customer.join(
            orders,
            on="CustId",
            how="inner"
        )
    )

    # Try to read existing ordersummary data
    try:
        existing_data = spark.table(f"{CATALOG}.{SCHEMA}.ordersummary")

        # Identify records that need to be updated (where customer data has changed)
        # ...

        # Combine unchanged records, expired records, new active records, and completely new records
        # ...

    except Exception as e:
        # If the table doesn't exist yet or there's an error, create initial data
        return (
            current_joined
            .select(
                current_joined["CustId"],
                current_joined["Name"],
                current_joined["EmailId"],
                current_joined["Region"],
                current_joined["OrderId"],
                current_joined["ItemName"],
                current_joined["PricePerUnit"],
                current_joined["Qty"],
                current_joined["Date"],
                current_joined["TotalAmount"],
                lit(True).alias("IsActive"),
                current_timestamp().alias("StartDate"),
                lit(None).cast(TimestampType()).alias("EndDate")
            )
        )
)

```

Step 5: Create Customer Aggregate Spend Table

This step creates the `customeraggregatespend` table, which aggregates customer spending by name and date.

```
@dlt.table(
    name="customeraggregatespend",
    comment="Aggregated customer spending by name and date",
    temporary=False
)
def customeraggregatespend():
    # Read from the ordersummary table
    ordersummary_data = dlt.read("ordersummary")

    # Only use active records for aggregation
    active_records = ordersummary_data.filter(col("IsActive") == True)

    # Aggregate TotalAmount by Name and Date
    return (
        active_records
        .groupBy("Name", "Date")
        .agg(sum_("TotalAmount").alias("TotalAmount"))
    )
```